IPC Testing and Test Cases

1. Purpose of IPC Testing

IPC testing was performed to verify that the **UI Process, Core Process and Logging Process** communicate correctly while running as independent processes.

The testing focused on verifying:

- Commands sent from the UI Process to the Core Process.
- Responses returned from the Core Process to the UI Process.
- Correct execution of CPU instructions after process separation.
- Transfer of execution and system log messages to the Logging Process.
- Correct operation of the Stack and FIFO Queue after integration.
- Correct handling of program termination.
- Successful storage of execution information in `simulator.log`.

---

2. IPC Test Cases

Test ID	Test Scenario	Input / Command	Expected Result	Actual Result	Status
IPC-01	UI–Core Communication	Connect UI to Core using TCP port 5001	UI and Core should establish communication	UI successfully communicated with Core	PASS
IPC-02	Program Loading	LOAD a valid assembly program	Core should receive and load the program	Core reported that the program was successfully loaded	PASS
IPC-03	CPU Reset	RESET	CPU should return to its initial execution state	CPU reset completed successfully	PASS
IPC-04	Single-Step Execution	STEP	Core should execute one instruction and update CPU state	One instruction executed and CPU state/PC updated	PASS
IPC-05	Continuous Execution	RUN using repeated STEP requests	Instructions should execute sequentially until HALT	Complete program executed successfully until HALT	PASS

IPC-06	Program Termination	HALT	CPU should stop further instruction execution	Core reported `CPU HALTED`	PASS
IPC-07	Core Response to UI	Execute simulator command	Core should return the updated simulator state	Updated CPU state was successfully displayed by UI	PASS
IPC-08	Core–Logger Communication	Generate execution events	Execution messages should reach the Logging Process	Core execution messages were successfully logged	PASS
IPC-09	Stack Operation	PUSH followed by POP	Stack should store and return the required value	PUSH and POP executed successfully	PASS
IPC-10	FIFO Queue Operation	ENQUEUE followed by DEQUEUE	FIFO Queue should insert and remove values correctly	ENQUEUE and DEQUEUE executed successfully	PASS
IPC-11	Log File Verification	Execute a complete program	Execution events should be stored in `simulator.log`	Execution information appeared successfully in `simulator.log`	PASS

---

3. Stack and FIFO Queue Integration Test

The following program was used to test CPU execution together with Stack and FIFO Queue operations:

```
MOV A,#15
MOV R0,#10
PUSH A
POP R1
ENQUEUE R0
DEQUEUE
ADD A,#1
HALT
```

This program tested the integration of different Core components after the simulator was converted into the multi-process architecture.

The operations verified were:

- `MOV A,#15` loads a value into the Accumulator.
- `MOV R0,#10` loads a value into register R0.
- `PUSH A` pushes the Accumulator value onto the Stack.
- `POP R1` removes the value from the Stack and stores it in R1.
- `ENQUEUE R0` inserts the value of R0 into the FIFO Queue.
- `DEQUEUE` removes an element from the FIFO Queue.
- `ADD A,#1` verifies continued CPU execution after Stack and Queue operations.
- `HALT` terminates program execution.

The Stack and FIFO Queue operations executed successfully during integrated testing.

---

4. Final End-to-End Test

A four-instruction program was executed to verify the complete multi-process simulator operation:

```
MOV A,#10
ADD A,#5
INC A
HALT
```

The expected execution sequence was:

Instruction	Expected Result
`MOV A,#10`	A = 10
`ADD A,#5`	A = 15
`INC A`	A = 16
`HALT`	CPU execution terminates

The final value of the Accumulator was **16**, confirming that the instructions were executed correctly.

At a high level, the test verified the following integrated operation:

UI Process → Core Process → CPU Execution

The Core Process also generated execution information that was transferred to the Logging Process:

Core Process → Logging Process → simulator.log

This confirmed that CPU execution and logging continued to operate correctly after the simulator was divided into independent processes.

---

5. Logging Evidence

During the final test, execution information was successfully recorded in `simulator.log`.

Important entries included:

```
[CORE] PROGRAM LOADED: 4 instructions
[UI] Response received from Core
[UI] Sending to Core: RESET
[UI] RESET
[CORE] CPU RESET
[UI] Sending to Core: STEP
[UI] STEP
[CORE] STEP started at PC=0
[CORE] STEP completed
[CORE] PC: 0 -> 1
[CORE] CPU STATE: A=10 B=0 SP=7
[CORE] CPU STATE: A=15 B=0 SP=7
[CORE] CPU STATE: A=16 B=0 SP=7
[CORE] CPU HALTED
```

These log entries provide evidence that:

- The program was successfully loaded.
- RESET was received and processed.
- STEP commands were executed.
- The Program Counter progressed during execution.
- CPU state changes were recorded.
- The Accumulator progressed from 10 to 15 and finally to 16.
- HALT was detected correctly.
- Execution information was successfully stored in the log file.

---

6. IPC Test Result Summary

The IPC and integration tests confirmed that the three-process simulator was functioning correctly.

The tests demonstrated that:

- The UI Process successfully communicates with the Core Process.
- The Core Process receives and executes simulator commands.
- Updated CPU state is returned to the UI.

- Programs can be loaded, reset and executed instruction by instruction.
- RUN operation can be performed through repeated STEP requests until HALT.
- Stack PUSH and POP operations remain functional.
- FIFO ENQUEUE and DEQUEUE operations remain functional.
- Execution information is successfully transferred to the Logging Process.
- The Logging Process successfully stores execution information in `simulator.log`.
- HALT correctly terminates CPU execution.

Therefore, the testing demonstrates that the simulator's main functionality continued to operate successfully after its transition from a standalone architecture to a **multi-process architecture using TCP socket-based IPC**.